Facultad de Defensa Nacional

Licenciatura en CIBERDEFENSA

Lenguajes de Programación

Introducción a la Programación Orientada a Objetos

v 1.0

Docentes:

Ing. Sebastián Balestrini

Esp. Ing. Gustavo Di Stefano

Carga horaria: 64 hs

Tabla de contenido

Tabla de contenido *1*

Introducción a la Programación Orientada a Objetos (POO) *2*

¿Qué es la Programación orientada a Objetos (POO)? **2**

Clases y Objetos **2**

Propiedades y métodos **4**

Comparación de paradigma estructurado y POO **5**

Introducción al UML **5**

Visibilidad en POO **6**

Atributos públicos (public) 6

Atributos públicos (private) 6

Atributos protegidos (protected) 6

Encapsulamiento **7**

Herencia y polimorfismo **8**

Ejemplos de clases en C++ **11**

Ejemplos de clases en Java **12**

Introducción a la Programación Orientada a Objetos (POO)

¿Qué es la Programación orientada a Objetos (POO)?

La **P**rogramación **O**rientada a **O**bjetos (**POO**) es un paradigma de programación que organiza el software en torno a objetos: unidades que combinan *estado* (datos) y *comportamiento* (métodos) bajo una misma entidad. La idea central es modelar el dominio de un problema trasladando conceptos del mundo real (o del negocio) a objetos de software que interactúan entre sí enviándose mensajes (llamadas a métodos).
A diferencia del enfoque puramente procedimental —donde el código se estructura como una secuencia de funciones que operan sobre datos externos— en **POO** los datos viajan con sus operaciones: cada objeto protege su propio estado y expone solo las acciones necesarias para interactuar, lo que favorece la abstracción y el encapsulamiento.

¿Por qué la POO es útil en la programación de aplicaciones? Porque facilita construir sistemas modulares y mantenibles: cada clase tiene una responsabilidad clara, el cambio en una parte del sistema impacta menos en el resto, y los objetos se pueden reutilizar o extender sin reescribir desde cero. Además, la POO permite expresar relaciones naturales del dominio (por ejemplo, “un Pedido tiene Ítems”, “un Usuario hereda de Persona”) y promueve prácticas como pruebas unitarias y separación de capas (presentación, dominio, persistencia).
Como repaso conceptual (viniendo de Java), la POO en cualquier lenguaje implica: pensar en tipos (clases), instancias (objetos), interfaces de uso (métodos públicos) y contratos de comportamiento (polimorfismo), priorizando composición sobre herencia cuando queremos flexibilidad.

Clases y Objetos

En POO, el concepto de clase es molde que define cómo se crean los objetos. Una clase especifica las propiedades (atributos o variables internas) y los métodos (funciones asociadas) que caracterizan a un conjunto de entidades.

Por ejemplo, podemos pensar en la **clase** Vehículo:

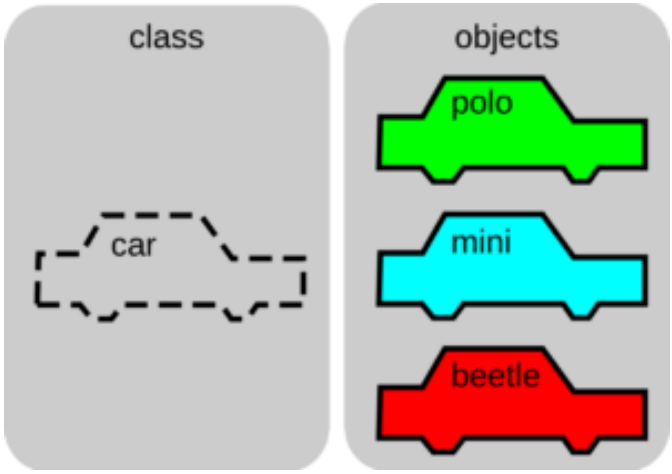
- Sus propiedades podrían ser color, velocidad, ruedas, motor, marca
- Sus métodos podrían ser arrancar(), frenar(), doblar().

Un **objeto**, en cambio, es una instancia concreta de una **clase**. Es decir, cuando a partir del molde “Auto” creamos un objeto en memoria, tenemos un auto particular con datos específicos:

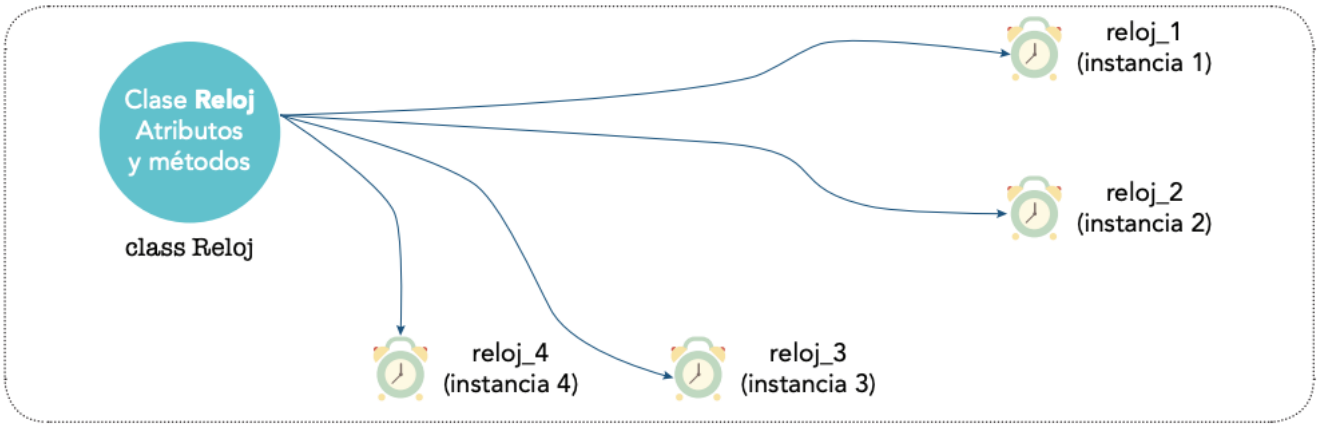
- Objeto 1: marca = "Toyota", modelo = "Corolla", color = "Rojo".
- Objeto 2: marca = "Ford", modelo = "Focus", color = "Azul".

Cada objeto tiene su propio estado (los valores de sus propiedades) y responde a las mismas operaciones definidas en la clase. Esta relación entre clases y objetos permite reutilizar código, ya que una sola definición de clase puede producir múltiples instancias distintas, todas compartiendo la misma estructura y comportamiento básico.

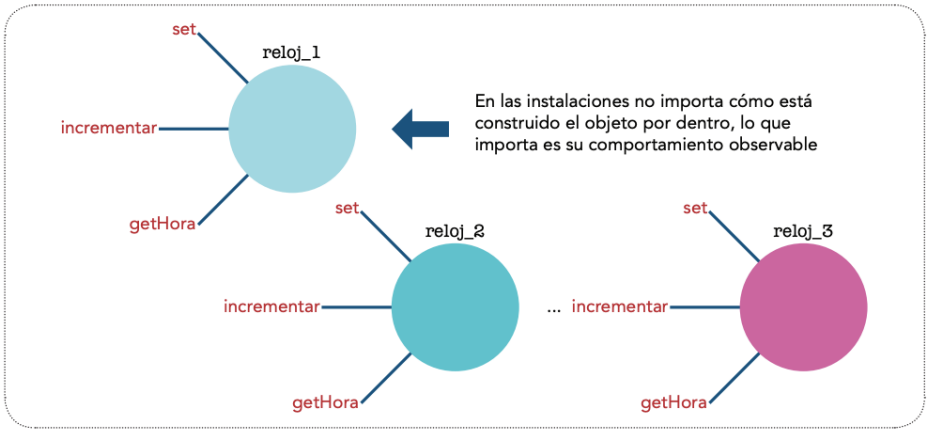
Una representación gráfica sencilla podría ser la siguiente:



Aquí vemos como la clase “car” se utilizó para construir objetos específicos (Autos) donde cada uno de ellos tiene un estado particular (datos específicos)
Otra representación sencilla podría ser el siguiente ejemplo:



Donde vemos claramente 4 relojes que parten de la misma clase pero son entidades completamente individuales donde solo comparten una estructura y comportamiento común



Propiedades y métodos

Dentro de una clase, encontramos dos componentes fundamentales: propiedades y métodos.

- **Propiedades**: Son las variables que almacenan el estado de un objeto. Cada instancia de la clase mantiene sus propios valores para esas propiedades. Por ejemplo, en una clase Persona, las propiedades podrían ser nombre, edad y dni. Así, el objeto persona1 puede tener nombre = "Juan" y edad = 25, mientras que persona2 puede tener nombre = "Ana" y edad = 30.
- **Métodos**: Son las funciones definidas dentro de una clase que describen su comportamiento. Representan las acciones que un objeto puede realizar. Por ejemplo, en la clase Persona, un método podría ser saludar(), que muestre un mensaje como “Hola, soy Juan”.

La combinación de propiedades y métodos es lo que da vida a un objeto:

- Las propiedades definen qué es.
- Los métodos definen qué puede hacer.

Comparación de paradigma estructurado y POO

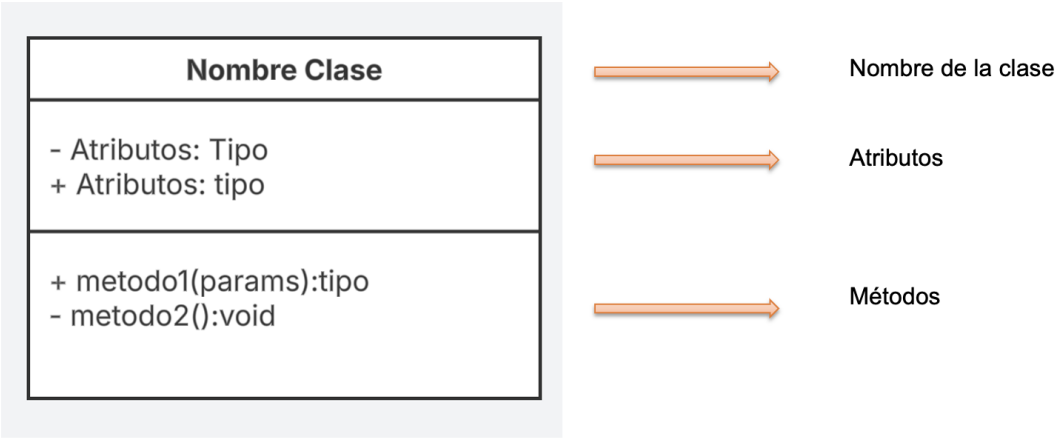
Estructurado	**POO**
Se basa en funciones	Se basa en clases y objetos
Datos y funciones separados	Datos y métodos juntos
Menos intuitivo para sistemas grandes	Más natural para modelar el mundo real
Ejemplo: C	Ejemplo: Java, C++

Introducción al UML

El Lenguaje Unificado de Modelado (UML, por sus siglas en inglés) es una herramienta gráfica utilizada en ingeniería de software para representar visualmente cómo se estructuran y relacionan los sistemas. Dentro de UML, uno de los diagramas más utilizados es el diagrama de clases, que nos permite representar propiedades y métodos de una clase de manera sencilla.

Un diagrama de clase básico se representa como un rectángulo dividido en tres secciones:

1. Nombre de la clase (parte superior).
2. Propiedades o atributos (parte intermedia).
3. Métodos o funciones (parte inferior).



Visibilidad en POO

En la Programación Orientada a Objetos, los atributos y métodos de una clase pueden tener distintos niveles de acceso, definidos por modificadores de visibilidad. Esta característica, conocida como encapsulamiento, permite controlar cómo y desde dónde pueden ser utilizados los datos internos de un objeto. El uso adecuado de visibilidad contribuye a mantener la integridad del objeto, promoviendo un diseño más seguro, modular y fácil de mantener.

Atributos públicos (public)

Los atributos públicos pueden ser accedidos directamente desde cualquier parte del programa, incluso desde fuera de la clase. Si bien su uso es sencillo, exponer directamente los atributos puede romper el principio de encapsulamiento, por lo que suele reservarse para constantes u objetos que no requieren protección.

Atributos públicos (private)

Los atributos privados solo pueden ser accedidos desde dentro de la propia clase. Es el nivel de protección más estricto y el más recomendado para proteger el estado interno de un objeto. Para manipular estos atributos desde fuera de la clase, se utilizan generalmente métodos públicos conocidos como getters y setters.

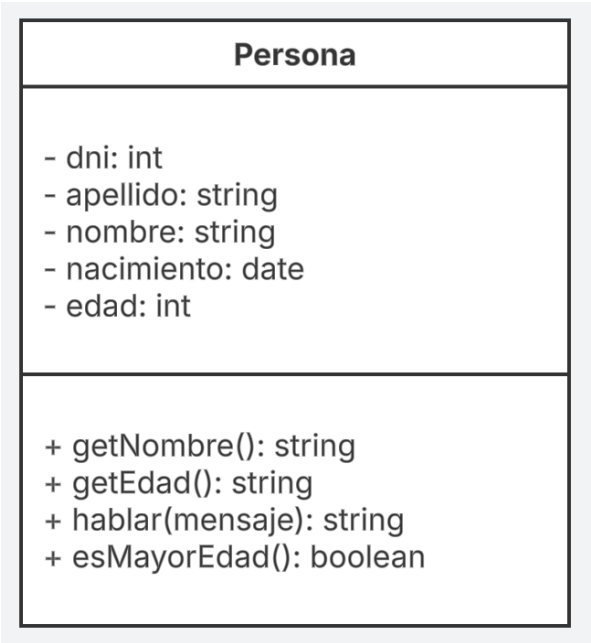
Atributos protegidos (protected)

Los atributos protegidos son similares a los privados, pero con la diferencia de que pueden ser accedidos también desde las subclases (clases que heredan de la clase base). Se utilizan comúnmente cuando se quiere mantener el encapsulamiento frente al exterior, pero permitir cierta extensión o modificación desde clases derivadas.

En UML esta visibilidad está representada de la siguiente manera:

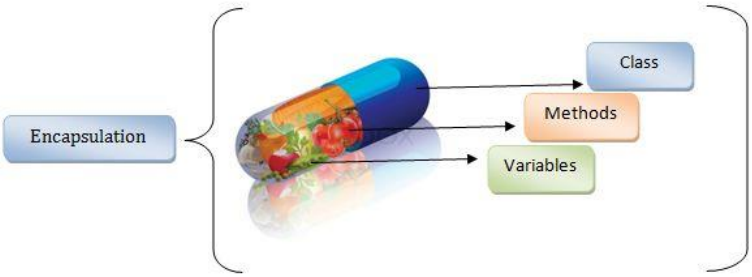
- + Publico
- - Privado
- # Protegido

Un ejemplo concreto de una clase Persona podría ser:



Encapsulamiento

El encapsulamiento es el principio de POO que establece que los datos internos de un objeto deben estar protegidos y solo ser accesibles a través de una interfaz controlada. La idea central es que un objeto no exponga directamente cómo funciona internamente, sino que ofrezca métodos públicos que permitan interactuar con él de manera segura.



Encapsulamiento:

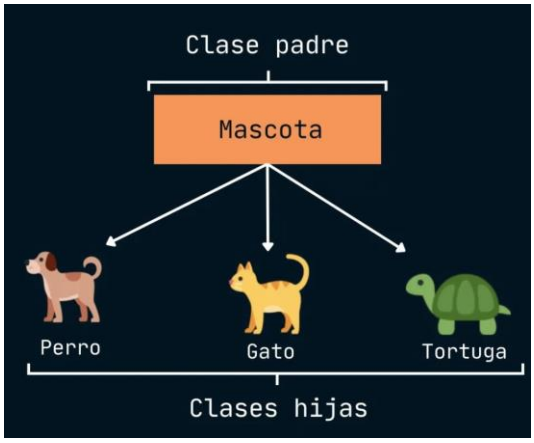
- Ocultar los detalles internos de una clase y exponer solo lo necesario.
- Protege los atributos usando modificadores de acceso.
- Permite mantener control sobre los datos.

Por ejemplo, en una clase **CuentaBancaria**, la propiedad saldo no debería poder modificarse directamente desde afuera del objeto. En cambio, el acceso al saldo se controla mediante métodos como **depositar**() o **retirar**(). De esta forma, la clase asegura que nunca se deposite un valor negativo o que no se pueda retirar más dinero del que está disponible.

El encapsulamiento favorece la seguridad, la coherencia de los datos y la mantenibilidad del código, ya que los detalles internos de la clase pueden cambiar sin afectar a quienes la utilizan.

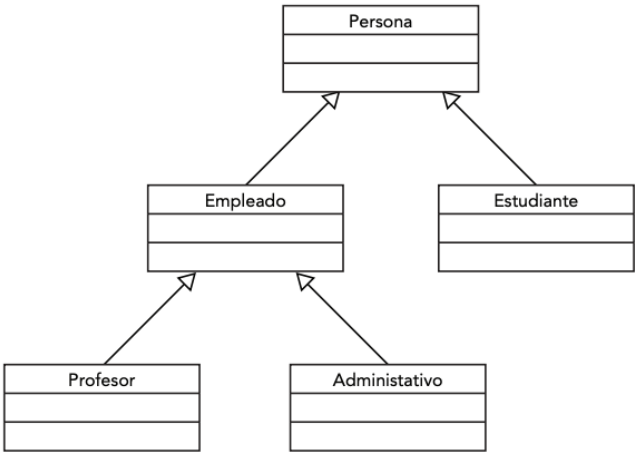
Herencia y polimorfismo

La **herencia** es uno de los pilares fundamentales de la **Programación Orientada a Objetos**. Permite que una clase (llamada clase hija o subclase) reutilice atributos y métodos de otra clase (llamada clase padre o superclase). Gracias a la herencia, no es necesario repetir código, ya que lo que está definido en la superclase pasa automáticamente a estar disponible en la subclase.



Por ejemplo, si tenemos una clase Persona con propiedades como nombre y edad, y métodos como saludar(), podemos crear una clase Estudiante que herede de Persona. De esta forma, Estudiante tendrá todas las propiedades y métodos de Persona, pero además podrá tener sus propios atributos (carrera, legajo) y métodos (inscribirse()).

La herencia en POO refleja relaciones del tipo “es un”. Un estudiante es una persona, por lo que heredar resulta natural.

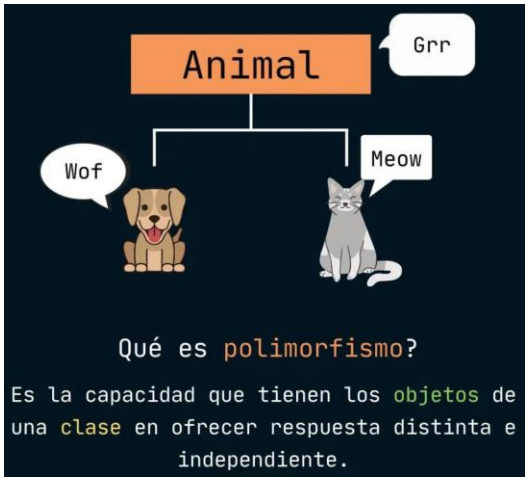


Características de la herencia:

- Una clase hija hereda atributos y métodos de una clase padre.

- Permite reutilizar código y extender comportamientos.

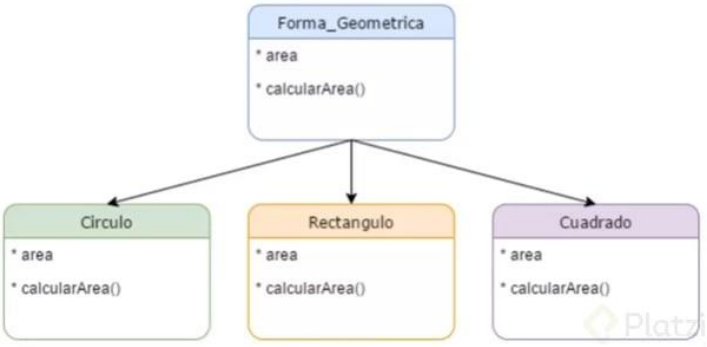
El **polimorfismo** complementa la herencia al permitir que una subclase sobrescriba (override) métodos definidos en la clase padre para adaptarlos a su propia lógica.



Por lo tanto:

- Un mismo método se comporta diferente según el objeto que lo use.
- Permite sobrescribir métodos en clases hijas

Supongamos el siguiente ejemplo:



Desde el punto de vista del código que los utiliza, ambos objetos responden al mismo mensaje (calcularArea()), pero el comportamiento concreto cambia según la clase que lo implemente.

Esto es el polimorfismo: un mismo mensaje puede producir diferentes comportamientos dependiendo del objeto que lo reciba.

Ejemplos de clases en C++

El siguiente código en C++ define una clase llamada Persona, la cual representa a una persona con tres atributos privados: nombre, apellido y edad. Estos atributos están encapsulados para proteger el acceso directo desde el exterior. La clase incluye un constructor público que permite inicializar una nueva persona con valores específicos para nombre, apellido y edad. Además, cuenta con un método llamado saludo(), que imprime por pantalla un mensaje personalizado con los datos de la persona. En la función main, se crea una instancia de Persona llamada p1, inicializada con los valores "Ana", "Gonzalez" y 29, y luego se invoca el método saludo() para mostrar su presentación. Este ejemplo ilustra los conceptos básicos de encapsulamiento, instanciación de objetos y definición de métodos en la Programación Orientada a Objetos con C++.

```cpp
#include <iostream>
#include <string>
using namespace std;

class Persona {
private:
    string nombre;
    string apellido;
    int edad;

public:
    // Constructor
    Persona(string nom, string ape, int e) {
        nombre = nom;
        apellido = ape;
        edad = e;
    }

    // Método saludo
    void saludo() {
        cout << "Hola, mi nombre es " << nombre << " " << apellido
             << " y tengo " << edad << " años." << endl;
    }
};

int main() {
    Persona p1("Ana", "Gonzalez", 29);
    p1.saludo();

    return 0;
}

```

Ejemplos de clases en Java

Este código en Java presenta la misma clase llamada Persona que en el ejemplo anterior en C++, pero implementada con la sintaxis y convenciones propias del lenguaje Java. Los atributos nombre, apellido y edad siguen siendo privados, asegurando el encapsulamiento. El constructor utiliza la palabra clave this para distinguir entre parámetros y atributos de instancia. El método saludo() cumple la misma función que en C++, pero usa System.out.println para imprimir en consola. Finalmente, la clase Main actúa como punto de entrada del programa, donde se instancia un objeto Persona y se invoca su método, reproduciendo exactamente el mismo comportamiento que el ejemplo en C++ pero dentro del paradigma de Java.

```java
class Persona {
    private String nombre;
    private String apellido;
    private int edad;

    public Persona(String nombre, String apellido, int edad) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.edad = edad;
    }

    public void saludo() {
        System.out.println("Hola, mi nombre es " + nombre + " " + apellido +
                           " y tengo " + edad + " años.");
    }
}

public class Main { // esta coincide con el archivo Main.java
    public static void main(String[] args) {
        Persona p1 = new Persona("Ana", "Gonzalez", 29);
        p1.saludo();
    }
}
```